Part 2: LLM agent for the bag battler (note)

Agent: `game/bots/llm.js` (final, v2) and `game/bots/llm_v1.js` (before the fix). Core in `game/bots/lib/agent.js`. **Reproduce:** see `README.md`. All games are logged in `game/logs/<bot>/` (run.js format, replayable in `index.html`); what the model saw and said is in `game/traces/<bot>/`. **[measured]** = from the runs. **[assumed]** = judgement.

1. What was built

- **Rule bot (baseline):** greedy. It tries every legal merge/place/move/buy on a copy of the game, scores the bag (damage per tick vs the next enemy and vs the boss, whether the exact simulator says the next fight is won, plus small credits for shields and heals), and takes the best action. Two rules came from its own early failures on practice seeds 5001–5100: buy weapons first while the next fight is predicted lost, and never fight a predicted loss while a reroll is affordable.
- **LLM agent:** one conversation per wave. The system prompt holds the exact rules and all item shapes per rotation. The state is text (bag drawn as a grid, links, damage, shop, next enemy, boss). Tools:
- `act(actions[], then_fight)`: runs actions in order, stops at the first illegal one, and returns the results, the new state and an exact battle preview;
- `preview_fight`;
- `fight`. `run.js` asks for one action at a time and synchronously, so act() actions are queued and each is validated on a copy of the game first; illegal ones go back to the model as errors. HTTP runs in a child process, so it is plain Node with no packages. Provider-agnostic (Gemini / Claude / Ollama). The plumbing is tested without any API by a fake model that plays like the rule bot through the same tools (`game/test/agent.test.js`).
- **Model: Gemini 3.5 Flash-Lite**, free tier, temperature 0. We had no paid key. Gemini Flash allows only 20 requests/day free; Flash-Lite allows 15/minute and **500/day** [measured from the API's error messages]. This forced the design: calls paced to 14/min, batched actions, previews inside act() results, and a quota stop that discards the interrupted game instead of counting it as a loss.

2. Results on the same 30 seeds (1–30) [measured]

bot	wins	win rate [95% CI]	avg wave reached	moves/game	LLM calls/game	tokens in/out per game	illegal actions/game	sec/game
random	0/30	0% [0, 11]	1.50	7.7	–	–	–	0.001
rule	25/30	83% [66, 93]	9.70	104	–	–	–	0.4
LLM v1	0/30	0% [0, 11]	3.53	32	11.4	52.0k / 1.2k	5.4	51
LLM v2	0/30	0% [0, 11]	3.77	46	19.1	115.4k / 1.8k	7.8	126

- v2's seeds 1–23 ran on the free tier; when its daily quota (500 requests) ran out, seeds 24–30 ran on a paid key with the same model and settings. Total cost of those 7 games: under €1.
- Rule bot over 200 seeds: 179/200 = 89.5% [84.5, 93.0]; random over 200: 0/200.
- **The LLM agent does not beat the rule bot.** Paired on 30 seeds: 0 vs 25 wins (McNemar $p < 0.001$), 6.2 fewer waves on average [5.4, 7.0].
- **It does beat random**, but not on wins (both 0). It survives 2.0 waves longer [1.2, 2.8], paired bootstrap.
- **Cost:** v1 used 341 calls, 1.56M input tokens and 35k output tokens for 30 games (25 min). On the free tier this cost $0. Input dominates because each call resends the conversation of the current wave. v2 used 574 calls, 3.46M input and 54k output tokens for 30 games (63 min).

3. Error analysis (5 lost games read by hand + counts over all 30) [measured]

Details: `part2/ERROR_ANALYSIS.md`.

class, in the build phase that lost the game	lost games of 30	example
illegal action in that wave (caught, reported, model retried)	23	re-buying an empty shop slot (42×), unknown id (30×), does not fit (25×)
fought blind into a predicted loss (`then_fight`, never read the preview)	**22**	seed 1: fought wave 4 at 14 HP; the preview showed death in 2 ticks
≥5 gold unspent when it lost	17	seed 8: forced to fight with 56 gold
no working weapon	11 (all wave-1 losses)	seed 3: soldiers only diagonal to the sword; seed 4: never bought a weapon
forced by the call cap	4	seed 8

Choice: fix the blind fight. - **It is the irreversible step.** The agent already had the exact answer (the simulator) and did not look. - **Unblocking it lets the agent repair the other mistakes:** buy a weapon, spend gold, power a weapon. - **Why not illegal actions,** though they are slightly more frequent: they are recoverable (caught and retried) and split into 8 different kinds, so no single change fixes them. - **Spatial mistakes** (about half the lost games) are the next candidate; that fix is prepared but unmeasured (`game/bots/lib/state_v2.js`).

The one change (tool), v2: `then_fight` only starts the battle if the simulator predicts a win. Otherwise the act() result says "NOT executed: you would LOSE" with the preview, and the model must change the bag or call `fight()` deliberately. Model, prompt, state text and seeds are unchanged.

4. Did the fix work? Is the difference bigger than noise? [measured, seeds 1–30, paired]

	v1	v2	v2 – v1 [95% CI]
wins	0/30	0/30	McNemar: no discordant seeds, $p = 1$

	v1	v2	v2 – v1 [95% CI]
avg wave reached	3.53	3.77	**+0.23 [–0.43, +0.90]**, paired bootstrap
seeds better / same / worse		9 / 14 / 7	
losing wave had a blind fight	22/30	**0/30**	
losing wave forced by the call cap	4/30	16/30	
LLM calls per game	11.4	19.1	+68%

- **The fix removed its target:** no lost game ended in a blind fight. The guard blocked 91 losing `then_fight` calls.
- **It did not improve the outcome:** +0.23 waves, with a CI that contains 0, so the difference is **not bigger than noise**. Wins stayed at 0.
- **Why:** after a fight was blocked, the model knew it would lose but could not build a winning bag. It either called `fight()` anyway, knowing the result (14 of 30, e.g. seed 21: *"Start battle with empty bag"*), or used up its 14 calls rearranging (16 of 30). Blindness was a symptom; the cause is that this model cannot plan the bag (adjacency, ids, shop state).
- **What that implies:** the next change should help building, not fighting: the prepared spatial-facts state (`game/bots/lib/state_v2.js`), or a stronger model. The guard is kept in v2 because it's cheap and only blocks provable losses, at the price of 68% more calls.
- **Run-to-run noise:** not measured separately (no quota left for a v1 rerun). The paired bootstrap already shows the difference is within the seed-to-seed variation.

5. What I do not trust / limitations

1. **A weak, cheap model.** The result is about *this* agent with Gemini 3.5 Flash-Lite. A stronger model (Gemini Pro, Claude Opus) would likely do much better; we could not test that for free.
2. **30 games is small.** A 95% CI for 0/30 still allows up to 11% true wins. Small improvements in win rate cannot be detected; average wave reached is the more sensitive measure.
3. **The rule bot is strong partly because it uses the exact simulator for every candidate action.** The LLM has the same simulator but uses it far less (once per act() call).
4. **Two API tiers for v2.** Seeds 1–23 ran on the free tier, 24–30 on a paid key: same model, settings and temperature, so this shouldn't matter, but it is a difference.
5. **Not perfectly reproducible.** Temperature 0 is not a guarantee, so their rerun on hidden seeds will differ somewhat even on the same seeds.
6. **Prompt and protocol were adjusted once before measuring** (after smoke tests, never on seeds 1–30): v1 at first fought without ever previewing, because the prompt said "use as few calls as possible". We softened it to "batch actions, but check the preview".
7. **What I cut, and why:** a repeat v1 run to measure run-to-run API noise (no free quota left; the paired bootstrap covers seed-to-seed noise), a stronger model (no paid key during the work), and a second improvement (the spatial-facts state is written but unmeasured, because the brief asks for one change at a time).

6. AI tools used and how I checked them

Claude Code (Opus 5.5) wrote the bots, harness, analysis and notes with me. I checked it through: - a fake-model test of the agent before spending any API calls; - reading game stories (`part2/story.js`) instead of trusting summary numbers; - making every claim in this note come from a script output; - questioning results that looked wrong. The rule bot's 30% wave-1 losses led to the weapon-first rule; the first "daily quota" stop was double-checked against the API's own message (500/day).